

Pond

Storage & Versioned State

DOCUMENT	Pond RFC 1 — Storage & Versioned State
VERSION	1.0
STATUS	Architecture-Frozen
DATE	July 2026
AUDIENCE	Implementers, contributors, systems architects
PHILOSOPHY	One copy. Infinite execution. Zero coordination unless necessary.

SUMMARY

This document formally specifies the Pond storage ABI, object lifecycle, and Versioned State — the fundamental primitive from which transactions, snapshots, branches, time travel, replication, and placement all derive. It is the first of five RFCs that together specify the Pond architecture in implementable form.

Every claim in this document belongs to one of three buckets: **Specified** (architectural, implementations must conform), **Implementation-defined** (architecture permits multiple approaches), or **Research** (acknowledged unsolved, future work). This discipline — borrowed from LLVM, Linux, Raft, TigerBeetle, and FoundationDB — is what makes the specification honest and implementable.

1. Introduction & Philosophy

Pond is a capability-oriented data runtime: a minimal core coordinating many execution strategies over a single versioned, immutable, one-copy object substrate, with everything staying local unless coordination is provably necessary. The architecture is not a clone of Spark, Flink, Databricks, or Iceberg. It is a distinct architectural direction whose organizing principle is the composition of execution capabilities over Versioned State.

1.1 Philosophy

One copy. Infinite execution. Zero coordination unless necessary.

These three clauses capture the discipline that governs every architectural decision in Pond. One copy means that exactly one artifact — immutable sealed objects in object storage — is canonical; everything else (catalogs, indexes, materialized views, the OPEN object log) is a cache rebuildable in bounded time. Infinite execution means that capabilities compose without limit: SQL, streaming, transactions, AI, vector, graph, and future workloads all run over the same substrate through declarative capability interfaces. Zero coordination unless necessary means that everything stays local — single-node, single-process, in-process — unless the system can prove that coordination across nodes is required for correctness. This is the discipline behind SQLite, Git, TigerBeetle, DuckDB, and every other system that became successful by remaining small.

1.2 The Seven Architectural Laws

The architecture is governed by seven non-negotiable laws. Every change to the architecture must strengthen at least one of them; any change that weakens one is rejected by default. The laws are listed here for reference; their full implications are woven through every section of this RFC and the four RFCs that follow.

#	Law	What it forbids
1	One canonical copy. Everything else is cache.	Any second durable representation (LTAP's Postgres+Parquet; Fluss's log+lake).
2	Storage is permanent. Execution is replaceable.	Baking any execution strategy into storage; making any backend uncircumventable.
3	The core must remain tiny.	Adding subsystems that do not replace existing ones; concept accretion.
4	Distribution is an optimization, not another architecture.	"Cluster mode" vs "embedded mode" as separate code paths.
5	Capabilities before products.	Hardcoding Kafka, GPU, vectors, AI, formats, or replication algorithms.
6	Mechanical sympathy — measurable.	Abstractions that hide hardware cost; slogans without budgets.

#	Law	What it forbids
7	The complexity budget.	Feature creep; subsystem accretion; changes that do not remove more than they add.

Table 1.1: The seven architectural laws. Every RFC and every PR must answer which law it strengthens and none that it weakens.

1.3 Scope of this RFC

This is RFC 1 of 5. It specifies the storage layer: the four storage syscalls that form the internal kernel ABI, the object lifecycle state machine, Versioned State as the fundamental primitive, object layout targets, versioning and compatibility rules, the storage-relevant subset of failure domains, chaos tests as architectural acceptance tests, the storage-layer invariants, performance budgets, and the dependency budget for the storage kernel. The other four RFCs cover the Planner and IR (RFC 2), the transaction and consistency model (RFC 3), the capability and placement model (RFC 4), and the operational architecture (RFC 5).

The reader of this document is expected to be a systems engineer comfortable with content-addressed storage (Git), log-structured systems (FoundationDB, Postgres WAL), and capability or plugin architectures (LLVM backends, Linux device drivers). No knowledge of DuckDB, Spark, or any specific lakehouse format is assumed, though familiarity with Apache Iceberg or Delta Lake will make the Versioned State section easier to read.

2. The 3-Bucket Discipline

Every claim in this RFC — and in every subsequent RFC — belongs to exactly one of three buckets. The discipline is borrowed from how mature systems specifications are written: LLVM separates the IR specification from backend-specific behavior; Linux separates the syscall ABI from driver implementation; Raft separates the consensus algorithm from log-structured storage; TigerBeetle separates the deterministic state machine from the replication protocol; FoundationDB separates the transaction protocol from storage-server implementation. Pond follows the same discipline.

Bucket	Meaning	Examples
Specified	Architectural. Implementations must conform. Two independent implementations of a Specified behavior must produce identical observable results.	The four storage syscalls. The object lifecycle state machine. The content-addressed DAG pattern. Versioning and compatibility rules. The storage-layer invariants.
Implementation-defined	The architecture permits multiple approaches. The implementation chooses one and documents it. Different implementations may make different choices without violating the architecture.	HLC vs TrueTime for timestamps. Raft vs storage-aware replication. Object size within 128 MB–1 GB. Root store backend (embedded Raft, FDB, etcd, Postgres).

Bucket	Meaning	Examples
Research	Acknowledged unsolved. Future work. Named honestly rather than pretended solved. Multi-year effort, like LLVM IR was.	Streaming semantics (exactly-once, watermarks, incremental MVs). Distributed execution (Exchange across nodes). Cross-backend cost-based planning. Storage-aware replication protocol.

Table 2.1: The 3-bucket discipline. Every claim in this RFC is in one of these buckets.

2.1 Why the discipline matters

Architectures die when they quietly assume solutions to research problems. Spark assumed a unified RDD abstraction would handle streaming — and ended up adding Structured Streaming as a separate subsystem. Databricks LTAP assumes sub-second Postgres-to-Iceberg sync — measured sync lag in production is minutes, not milliseconds. Materialize spent years honestly acknowledging that differential dataflow was the hard research problem. The 3-bucket discipline forces the architect to say, explicitly, "this is specified," "this is impl-defined," or "this is research." It prevents the quiet assumption that destroys architectures years later.

2.2 How to read this RFC

When this RFC says a behavior is **Specified**, two independent implementations must produce identical observable results, or one of them is non-conformant. When it says a behavior is **Implementation-defined**, the implementation must document its choice and the trade-offs it implies. When it says a behavior is **Research**, the architecture permits a future solution but does not claim one exists today. Implementers who ignore this distinction and pretend a Research item is Specified will produce systems that fail in production for reasons the architecture already acknowledged.

3. The Storage ABI — Four Syscalls

The Pond storage kernel exposes exactly four operations. These are the internal kernel ABI — the equivalent of Linux syscalls. They are not the public SQL API. Users of Pond see CREATE TABLE, INSERT, SELECT, CREATE PIPESOURCE, CREATE INDEX, CREATE BRANCH. The four syscalls are the layer below SQL: every user-facing operation compiles down to a graph of these four primitives, executed by the Coordinator against the storage kernel. The surface is irreducible: fewer than four primitives cannot express a named, content-addressed, immutable database. More than four would either be library code (Collect is graph traversal via Read+Reference) or leak execution concepts into storage.

3.1 The four operations

Read

```
Read(hash_or_name) -> Bytes

// If given a content hash (e.g. sha256:abc123...), return the
// bytes of the sealed object with that hash.
// If given a name (e.g. "lake.events"), resolve the name to
// its current commit hash via the root pointer namespace,
// then return the bytes of the object at that hash.

// Precondition: hash exists OR name is bound in the root
// pointer namespace of the current database.
// Postcondition: returned bytes match hash(hash) exactly,
// or raise NOT_FOUND.
// Consistency: linearizable for name resolution (a name
// always resolves to its most-recently-committed hash);
// content-addressed reads are by construction consistent
// (a hash is a fixed value).
```

Write

```
Write(bytes) -> FragmentHandle

// Append a fragment (a batch of rows encoded as Arrow IPC,
// or a format-plugin-defined OPEN format) to the OPEN object
// of the current transaction on the current shard.
// Returns a FragmentHandle that includes the open_hash, the
// fragment offset, and the LSN (log sequence number) at which
// the fragment was replicated.

// Precondition: an OPEN object exists for the current
// transaction on the current shard. If not, the runtime
// creates one implicitly on the first Write.
// Postcondition: the fragment is fsync'd to the Raft log
// of the shard before the call returns. The fragment is
// visible to subsequent Reads of the OPEN object.
// Consistency: linearizable. A Write is acknowledged only
// after Raft quorum fsync.
```

Seal

```
Seal(open_hash) -> SealedHash

// Convert the OPEN object identified by open_hash into a
// SEALED object. The OPEN object's Arrow IPC (or other OPEN
// format) byte stream is converted to a Parquet file on
// object storage, the final content hash is computed, the
// metadata object (stats, bloom, pk_range, lineage, schema_id,
// branch_id, txid, ttl, checksums) is written, and the object
// is marked SEALED.

// Precondition: open_hash refers to an OPEN object whose
// replication is complete (all fragments acked by Raft
// quorum).
// Postcondition: the OPEN object is no longer writable. The
// returned SealedHash is the content-addressed hash of the
// sealed Parquet bytes plus the metadata object. The OPEN
// object's Raft log prefix up to the seal point is
// eligible for truncation (it is now derivable from the
// sealed Parquet).
// Consistency: idempotent. Sealing an already-sealed object
// returns the same hash.
```

Reference

```
Reference(name, hash) -> ()

// Set a mutable name -> hash mapping in the root pointer
// namespace. This is the only mutable operation in the storage
// kernel. Names are scoped to a database; hashes are global
// (content-addressed).

// Precondition: hash exists (the referenced object has been
// sealed). The caller has write permission on the name's
// namespace.
// Postcondition: subsequent Read(name) returns the bytes of
// the object at hash. The previous binding (if any) is
// preserved in the commit DAG as the parent of this
// reference update.
// Consistency: linearizable. A Reference update is
// acknowledged only after Raft quorum fsync on the root
// pointer store.
```

3.2 What was eliminated: Collect

An earlier draft of this ABI included a fifth operation, `Collect(predicates)`, which listed hashes matching given predicates. It has been eliminated. `Collect` is graph traversal — it is implementable as a library function over `Read+Reference` (walk the commit DAG, filter by predicate). Git has no `Collect` syscall; git log walks the DAG. Making `Collect` primitive would have leaked query concepts into storage. The four operations above are the irreducible minimum for a named, content-addressed, immutable database: `Read` to access data, `Write` to create data, `Seal` to make data immutable, `Reference` to name data.

3.3 Failure modes (Specified)

Each syscall has specified failure semantics. The implementation chooses how to detect and recover; the semantics are architectural.

Failure	Read	Write	Seal	Reference
Network partition (leader isolated)	Stale reads from followers; linearizable reads block	Blocks on majority side; unavailable on minority	Blocks (requires quorum)	Blocks (requires quorum)
Disk full (local NVMe)	Continues from S3	Blocks; backpressure to client	Blocks	Blocks
S3 unavailable	Continues from NVMe cache (bounded)	Continues (OPEN objects stay on NVMe)	Blocks (cannot write Parquet)	Continues (root store is local/Raft)
Quorum lost	Stale reads from S3	Unavailable for that shard	Unavailable for that shard	Unavailable for that namespace
Object not found	Raises NOT_FOUND (deterministic)	n/a	Raises INVALID_OPEN_HASH	Raises HASH_NOT_FOUND

Table 3.1: Failure semantics of each syscall. The semantics are architectural; recovery protocols are impl-defined.

3.4 Consistency guarantees (Specified)

Linearizable writes. Write, Seal, and Reference are acknowledged only after Raft quorum fsync on the relevant shard. A successful write is durable across quorum failures. An unacknowledged write may or may not be durable; the client must retry on timeout.

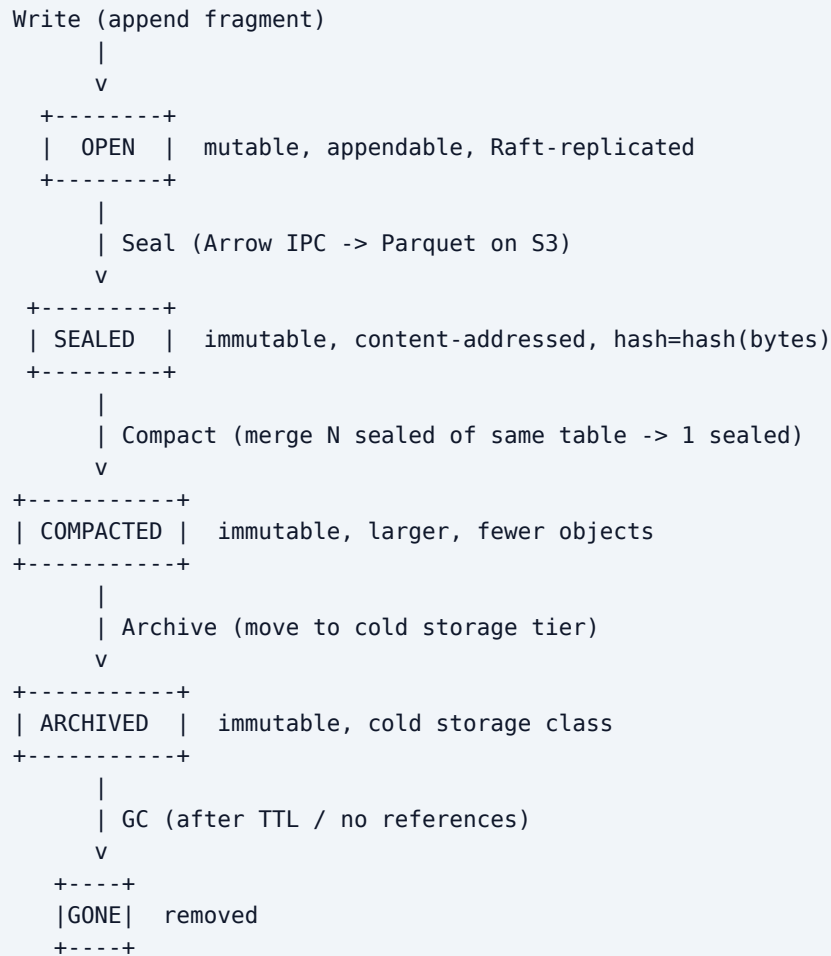
Snapshot reads. Read at a content hash is by construction consistent — the hash refers to fixed bytes that never change. Read at a name resolves the name to its current commit hash via the root pointer namespace, then reads at that hash. The snapshot is the commit hash; once resolved, the read is immune to concurrent commits.

Cross-region reads. Async Raft followers in remote regions serve reads at their last-applied commit hash. Read-your-writes within the primary region; eventually consistent (bounded by replication lag) across regions. This is the FoundationDB / Spanner / TigerBeetle pattern — the proven trade for single-region linearizability without specialized hardware.

4. Object Lifecycle State Machine

Every object in Pond transitions through a fixed lifecycle. The state machine is Specified: the states, the allowed transitions, and the invariants per state are architectural. The triggers for transitions (when to seal, when to compact, when to archive, when to gc) are Implementation-defined: the implementation chooses policies, within the constraints of the invariants.

4.1 The state machine



4.2 Invariants per state (Specified)

State	Mutability	Replication	Invariants
OPEN	Mutable. Appends allowed via Write.	Raft-replicated fragment log on NVMe. Bounded by drain lag.	Fsync'd to Raft quorum before Write ack. Rebuildable from sealed objects + log tail in O(un-sealed ops). Truncated after Seal.
SEALED	Immutable. No writes allowed.	Object storage (S3/GCS/Azure). Content-addressed.	Hash = hash(bytes), forever. Readable by any future Pond version. Drop and re-fetch from S3 if local cache lost.
COMPACTED	Immutable. Same content, different layout.	Object storage.	Logically equivalent to source sealed objects (same rows, possibly different physical layout). Lineage preserved in metadata.
ARCHIVED	Immutable. Moved to cold tier.	Cold storage class (S3 Glacier, etc.).	Readable but slower (minutes-hours latency). Restorable to warm tier on demand.
GONE	Removed.	n/a.	All references removed. Hash is dead. Cannot be resurrected.

Table 4.1: Invariants per lifecycle state. The state machine is architectural; transition triggers are impl-defined.

4.3 Transition triggers (Implementation-defined)

The implementation chooses when to trigger each transition, within the constraints of the invariants. Reasonable defaults:

Seal: when the OPEN object exceeds a size threshold (default 512 MB) or an age threshold (default 60 seconds), or when the transaction commits. Seal is also triggered by backpressure when the Raft log exceeds a configurable depth.

Compact: when the count of small sealed objects for a table exceeds a threshold (default 64). Bin-packing compaction is the v1 default; tiered compaction (RocksDB-style) is a research target.

Archive: when an object has not been read in N days (default 30). Driven by the placement capability, not the storage kernel itself.

GC: when an object has no live references and its TTL has expired (default 90 days after last reference dropped). GC is conservative: an object is only removed when the root pointer namespace, all branches, and all MV lineage commits agree it is unreferenced.

5. Versioned State — The Fundamental Primitive

Versioned State is the fundamental primitive of Pond. Everything else — transactions, snapshots, branches, time travel, replication, placement — is a derived semantic or policy over Versioned State. Earlier drafts of the architecture had transactions and placement as separate permanent concepts; elevating Versioned State and demoting them to derived semantics was the conceptual move that made the architecture coherent.

5.1 Definition

Versioned State is the union of three things: (1) the content-addressed DAG of immutable objects, (2) the object lifecycle state machine defined in section 4, and (3) the root pointer namespace that names versions. From these three, every other versioning concept derives:

Concept	How it derives from Versioned State
Snapshot	A Read at a specific commit hash. The hash refers to immutable bytes; the snapshot is by construction consistent.
Branch	A named pointer to a commit hash. Creating a branch is a Reference operation; the branch shares all sealed objects with its parent (copy-on-write semantics).
Tag	A named pointer to a commit hash, identical to a branch but conventionally immutable. Implemented as a Reference.
Time travel	A Read at a past commit hash. The commit DAG preserves all historical hashes; reading at any of them returns the state at that point.

Concept	How it derives from Versioned State
Transaction	A sequence of Write/Seal/Reference operations bounded by BEGIN and COMMIT. The COMMIT creates a new commit object whose parent is the previous commit. Specified in RFC 3.
Replication	A policy over Versioned State: which sealed objects are replicated where, and how the root pointer namespace is kept consistent across regions.
Placement	A policy over Versioned State: which sealed objects live on which storage tier (NVMe, S3 Standard, Glacier) and which region. Specified in RFC 4.
Rollback	A Reference operation that points a name to a previous commit hash. No data is destroyed; the current commit simply becomes unreferenced (and eventually GC'd).

Table 5.1: How every versioning concept derives from Versioned State.

5.2 The content-addressed DAG (Specified)

The DAG follows the Git model. There are four object types. Each is content-addressed: the hash of an object is the hash of its serialized bytes. The four types are:

Type	What it holds	Analogy
blob	Raw bytes — a Parquet file, an Arrow fragment, an index, a stats block. Opaque to the DAG.	Git blob
tree	A directory mapping {name -> hash}. A table is a tree of blob refs. A schema is a tree of column defs. An index is a tree of (key -> blob+offset) refs.	Git tree
commit	A snapshot pointer: {tree_hash, parent_commit_hash, author, timestamp, message}. A table at a point in time is a commit.	Git commit
tag	A named pointer to a commit: main, v2, agent_exp_42, events_per_min_mv. Branches, versions, MVs, named snapshots are all tags.	Git tag

Table 5.2: The four object types. Pattern over Read/Write/Seal/Reference, not separate syscalls.

These four types are patterns over the four syscalls, not separate primitives. A blob is Seal(Write(bytes)). A tree is Seal(Write(serialized_references)). A commit is Seal(Write(tree_hash + parent_hash + metadata)). A tag is Reference(name, commit_hash). The DAG is a convention for organizing objects; the storage kernel does not know about blob/tree/commit/tag as distinct types. This is the same discipline as Git, where the object model is a convention layered on top of a content-addressed blob store.

5.3 The root pointer namespace (Specified)

The root pointer namespace is the only mutable metadata in Pond. It maps names to commit hashes: lake.events -> commit_abc123, lake.users -> commit_def456, branch:exp -> commit_ghi789. Without it, you would need to know hashes to read anything — that is IPFS, not a database. Databases need names.

The root pointer namespace is small, mutable, and strongly consistent. It is replicated via its own Raft group (or, alternatively, backed by an external KV store like FoundationDB or etcd — impl-defined). Every other piece of metadata — commits, trees, schemas, indexes, stats, permissions — is content-addressed and immutable, stored as sealed objects on S3.

5.4 The metadata model (Specified)

Layer	What it stores	Where	Consistency
Root pointers	name -> commit_hash (small, KB-MB per database)	Root Store (Raft or external KV)	Linearizable
Commits	snapshot metadata (tree hash, parent, timestamp)	S3 (immutable, content-addressed)	Content-addressed
Trees	file/index references	S3 (immutable, content-addressed)	Content-addressed
Schemas	column definitions, types	S3 (immutable, content-addressed)	Content-addressed
Indexes	key -> blob+offset mappings	S3 (immutable, content-addressed)	Content-addressed
Stats	per-file column min/max/bloom	S3 (immutable, content-addressed)	Content-addressed
Permissions	ACL trees (principal -> capability)	S3 (immutable, content-addressed)	Content-addressed

Table 5.3: The metadata stack. Only root pointers are mutable; everything else is content-addressed.

This is why the architecture scales: only the root pointer layer requires coordination, and it is tiny. Everything else is content-addressed and naturally distributed — any node can cache any sealed object, any node can serve any read, and consistency is by construction (a hash is a fixed value). This is the same pattern as Git (most refs are content-addressed; HEAD is a tiny text file), Nix (store paths are content hashes; the manifest is small), and OCI (image layers are content-addressed; the manifest is a small JSON document).

5.5 The recursion proof

The catalog itself is a DuckLake-style table on S3 — its rows are metadata, its hot portion is cached in DuckDB memory, its cold portion is Parquet on S3. The catalog's own metadata (the catalog's catalog) is tiny and lives in the root pointer namespace. Recursion terminates cleanly at the root store. There is no infinite regress: at the bottom of the stack is a small strongly-consistent namespace, and above it is layered content-addressed immutable metadata.

5.6 State Views (future RFC)

The reader will notice that snapshot, branch, tag, time travel, rollback, and read-timestamp are all "views" over Versioned State. A future RFC may formalize State Views as a unified abstraction: a declarative way to specify a view (snapshot at hash H, branch B as of time T, etc.) that the planner can reason about. This is not specified in RFC 1; it is flagged here as a likely RFC 2 or RFC 5 topic. The current specification handles each view type as a derived semantic over Versioned State (per Table 5.1), which is sufficient for implementation.

6. Object Layout (Implementation-Defined)

Object layout — the physical size and structure of sealed Parquet files — is Implementation-defined. The architecture permits a range of choices; the implementation chooses within that range and documents its defaults. The targets below are the v1 defaults; they are subject to change based on benchmark results. They are not architectural guarantees.

Parameter	v1 default	Range	Trade-off
Sealed object size	512 MB	128 MB – 1 GB	< 128 MB: small-files problem (S3 LIST cost, metadata explosion). > 1 GB: scan skew, memory pressure, compaction cost.
Row group size	128 MB	64 MB – 256 MB	Parquet default. Larger groups improve sequential scan; smaller groups improve point-lookup skip.
Column chunk size	8–16 MB	4–32 MB	Affects memory pressure during scan; must fit comfortably in L2.
Compression	ZSTD level 3	ZSTD 1-19, LZ4, Snappy	ZSTD-3 is the Pareto-optimal point for analytical workloads. Higher levels improve ratio but cost CPU.
Sparse index granularity	1 entry per 1M rows	1 per 100K – 10M rows	Smaller granularities speed point lookups but inflate the index. ~1 KB per million rows is the ClickHouse default.

Table 6.1: Object layout parameters (impl-defined).

6.1 Why these ranges

The 128 MB–1 GB range for sealed object size is not arbitrary. Iceberg targets 512 MB; Delta targets 1 GB; Paimon targets 256 MB. Pond defaults to 512 MB (the Iceberg default, which has the most production validation) and is configurable per table. Tables with high write throughput may benefit from smaller sealed objects (faster seal cycles); tables with heavy analytical scan workloads may benefit from larger ones (fewer metadata entries, better sequential IO).

6.2 Format neutrality (Specified)

No storage format is privileged. Parquet is the v1 default for sealed objects; Arrow IPC is the v1 default for OPEN objects. Both are pluggable via the formats/ plugin taxonomy. A future implementation could use Lance (for ML workloads), Vortex (for newer columnar compression), or a custom format without changing the storage kernel. The kernel only requires that sealed objects are content-addressed and immutable — the format is the plugin's concern.

6.3 Interop with Iceberg and Delta (Specified)

Pond-native tables (DuckLake-style) have the cleanest one-copy story: writes go through the four syscalls, sealed objects are Parquet on S3, the catalog is content-addressed metadata on S3. Iceberg and Delta tables are read-mostly interop: Pond can read them via the formats/iceberg.so and formats/delta.so plugins. Writes to Iceberg are supported via the DuckDB iceberg extension (v1.4+, merge-on-read, positional deletes only). Writes to Delta are not yet supported upstream in DuckDB; this is a gap that may close in future DuckDB releases. For workloads that require Delta writes today, the recommended path is to write to a Pond-native table and mirror to Delta via a sink plugin.

7. Versioning & Compatibility Rules

Architectures die when versioning is vague. LLVM succeeded because IR stability exists — IR generated a decade ago still runs on modern LLVM. Git succeeded because object format stability exists — commits made in 2005 are still readable today. Linux succeeded because syscall stability exists — binaries compiled for Linux 2.0 still run on Linux 6.x. Without compatibility rules, every release becomes painful, users stop upgrading, and the project fragments. Pond specifies compatibility rules for every versioned artifact.

7.1 The versioning matrix (Specified)

Artifact	Version source	Compatibility rule
Storage ABI (4 syscalls)	Semantic versioning	Additive only. New syscalls allowed in minor versions; existing syscalls never change signature or semantics. Like Linux syscalls.
Object format (sealed blobs)	Schema-id in object metadata	Forward-compatible. New readers handle old objects; old readers skip unknown fields. Like Parquet.
Pond IR	IR version number in plan	Forward + backward compatible within major version. Old IR runs on new runtime; new runtime can read old IR. Major version bumps are explicit migration events. Like LLVM IR.
Schema (table columns)	Schema-id in commit metadata	Additive. Add columns, widen types. Destructive changes (drop, rename) require a new schema-id; old commits keep old schema. Like Iceberg schema evolution.
Capability declarations	Capability spec version	Plugins declare which spec version they target. Runtime refuses incompatible plugins at load time.
Catalog / metadata	Commit hash	Always content-addressed. Old commits are forever readable. No backward or forward compatibility issue — the hash is the version.
Wire protocols (pgwire, Quack)	Protocol version	Standard protocol versioning. Backward compatibility required for any protocol that ships as v1.

Table 7.1: Compatibility rules for every versioned artifact.

7.2 The stability promise (Specified)

Stability promise. Any object sealed today is readable by any future version of Pond. Any IR generated today is executable by any future runtime within the same major version. Any capability plugin built against spec version N will load on any runtime that supports spec version N or later. These are not aspirations; they are release criteria. A release that violates any of them is a regression and is blocked from shipping.

7.3 Migration events (Specified)

When a major version bump is required (e.g., Pond IR v2, which changes operator semantics incompatibly), the migration is explicit: a tool reads old IR and emits new IR; old sealed objects remain readable (object format stability is separate from IR stability). The migration tool is shipped with the new release. Users are not forced to migrate; old IR continues to run on old runtimes. This is the LLVM model: IR upgrades are opt-in, gradual, and reversible.

8. Failure Domains (Storage-Relevant Subset)

Production systems are defined by their failure domains. The failure domain matrix below specifies, for each storage-relevant failure mode, what continues to work and what blocks. The full failure domain matrix (including Coordinator, Planner, and capability failures) is in RFC 5. The semantics here are Specified; the recovery protocols are Implementation-defined.

Failure	Reads	Writes	Streaming	Recovery
S3 unavailable (whole region)	Continue from NVMe cache (bounded by cache size)	Block (cannot seal)	Continue if OPEN objects fit in NVMe	S3 recovery; replay OPEN logs
S3 degraded (elevated latency)	Continue (slower)	Continue (slower)	Continue (slower)	n/a
S3 object not found (hash mismatch)	Raise NOT_FOUND (deterministic)	n/a	n/a	Re-fetch from replication backend
Raft quorum lost (for one shard)	Continue from S3 (stale)	Block for that shard	Block for that shard	Quorum recovery or manual intervention
Root store quorum lost	Continue from cached roots (stale)	Block (all writes)	Block (new commits)	Quorum recovery
Network partition (minority side)	Continue (stale) from S3	Block on minority side	Block on minority side	Heal -> Raft catch-up
Whole node crash	Other nodes continue	Other nodes continue (Raft)	Other nodes continue	Rebuild from S3 + replay OPEN log
Disk full (local NVMe)	Continue from S3	Block; backpressure to client	Block	Compaction, then resume

Table 8.1: Storage-relevant failure domains. Full matrix in RFC 5.

8.1 The single-region linearizability trade

Pond is single-region linearizable. Cross-region is read-your-writes via async Raft followers + S3 cross-region replication. This is the FoundationDB / Spanner / TigerBeetle pattern: strong consistency in the primary region; eventual (bounded by replication lag) consistency across regions. Globally-linearizable OLTP without specialized hardware (TrueTime, atomic clocks) does not exist; Pond does not claim it. Workloads that require globally-linearizable OLTP should use CockroachDB or Spanner and CDC into Pond.

8.2 The S3 latency floor (fundamental)

Fundamental constraint. Truly cold PK lookups — keys not in any NVMe index — are 5 30 ms. This is the S3 GET floor and cannot be tuned away. The architecture mitigates this via NVMe-persistent sparse indexes (100 500 us for indexed keys), but a key not in any index will pay the S3 GET cost. Workloads that require sub-millisecond cold lookups for all keys should use TigerBeetle alongside Pond for hot keys.

9. Chaos Tests as Architectural Acceptance Tests

Every failure mode in section 8 must be executable as an automated chaos test. Chaos tests are not nice-to-haves; they are release blockers. A release that cannot demonstrate, via automated test, that it survives each failure mode does not ship. This is the discipline that separates production systems from research prototypes.

9.1 Required chaos tests (Specified)

Test	What it does	What it verifies
kill_coordinator	Kill the Coordinator process mid-transaction.	Writes continue (single-shard via Raft); new Coordinator elected; in-flight txn_records resolved.
lose_s3	Make S3 unavailable mid-write.	NVMe cache serves reads; writes block (not lose data); OPEN objects survive on NVMe.
kill_placement	Unload the placement capability.	Falls back to S3 default placement; reads continue; writes continue without NVMe promotion.
kill_gpu_backend	Make the GPU backend unavailable.	Planner re-routes OLAP queries to DuckDB; queries succeed (possibly slower).
kill_index_plugin	Unload the sparse index plugin.	Planner falls back to ART (DuckDB default); point lookups succeed (slower).
partition_minority	Network-partition the minority side of a 3-node cluster.	Minority unavailable; majority continues; on heal, Raft catches up.
crash_node	Hard-crash a node (kill -9).	Other nodes continue; rebuild from S3 + replay OPEN log; cold restart < 30 s at 1 PB.
root_store_quorum_loss	Lose root store quorum.	Writes block; reads continue from cached roots; on quorum recovery, writes resume.
seal_lag_burst	Sustained write burst exceeding drain throughput.	Backpressure engages; writes block (not lose data); no unbounded memory growth.

Table 9.1: Required chaos tests. Each is a release blocker.

9.2 How chaos tests are run

Chaos tests run in CI on every PR that touches the storage kernel, the Coordinator, or any capability that declares a fallback. A PR that fails a chaos test is blocked. The tests are deterministic: they seed the cluster with a fixed dataset, inject the failure at a fixed point, and verify the post-failure state matches the expected state. Non-determinism in chaos tests is a bug, not a feature — flaky chaos tests are fixed before any other work proceeds.

10. Architectural Invariants (Storage)

The storage layer maintains the following invariants. Each is Specified: any implementation that violates an invariant is non-conformant. The invariants are formally stated; the proofs are sketched. A complete formal proof is the subject of a separate document.

#	Invariant	Statement
1	One-copy	Only SEALED blobs on object storage are canonical. Every other artifact (OPEN object log, catalog cache, index cache, MVs) is rebuildable in bounded time from sealed blobs.
2	Immutability	Sealed objects never change. The hash of a sealed object is the hash of its bytes, forever. A sealed object cannot be modified, only superseded by a new sealed object.
3	Bounded recovery	After crash, the system recovers to the last-sealed state in $O(\text{un-sealed operations})$, never $O(\text{total historical data})$. Cold restart is bounded by the size of the OPEN object log, not by the size of the dataset.
4	Snapshot consistency	Any Read at a commit hash returns a consistent snapshot. Concurrent writes do not affect a read once the read has resolved the name to a hash.
5	Reference locality	Name resolution is deterministic for a given commit. The same name resolves to the same hash for all readers at the same commit; cross-region readers may see a stale commit, but the resolution is still deterministic at that commit.
6	Cache correctness	Cache invalidation is never stale for committed state. A reader never sees a cached value that has been superseded by a committed write. The cache may be stale (showing an older commit) but never inconsistent (showing a state that never existed).
7	Deterministic apply	Given the same DAG state and the same query, any conformant runtime produces the same result, regardless of which plugins are loaded. Non-determinism (<code>now()</code> , <code>random()</code> , <code>uuid()</code>) is resolved at the adapter layer before the storage kernel sees the operation.
8	Content-addressed integrity	The hash of a sealed object is computed from its bytes. Any bit-rot or corruption is detectable by re-hashing. Mismatched hashes raise <code>INTEGRITY_ERROR</code> and the object is re-fetched from the replication backend.

Table 10.1: Storage-layer invariants. Each is Specified.

11. Performance Budgets (Storage)

Performance budgets are targets to validate by benchmark, not architectural guarantees. The architecture permits these targets; the implementation must demonstrate them. A release that misses a target by more than $2\times$ is investigated; a release that misses by $10\times$ is blocked. The targets are anchored to measured numbers from DuckDB, DuckLake, pg_ducklake, and dragonboat research.

Operation	Target	Mechanism	Anchored to
Commit (single-shard, LAN)	< 1 ms	Raft quorum fsync on NVMe	dragonboat: 100K+ logs/sec
Commit (cross-AZ)	1–2 ms	Cross-AZ Raft round-trip	Standard cross-AZ RTT
Cross-node visibility	1–2 ms	Async Raft followers	dragonboat replication
Streaming freshness	1–2 ms	Tail-read OPEN object log	Local NVMe read
PK lookup (memory hit)	5–30 μ s	In-memory ART index	DuckDB ART (v0.4.1+)
PK lookup (NVMe index hit)	100–500 μ s	NVMe-persistent sparse index	ClickHouse sparse index
PK lookup (truly cold)	5–30 ms	S3 GET one Parquet row group	S3 Standard GET p50
OLAP scan (fresh data)	$\approx$ zero union tax	DuckDB unions SEALED + OPEN	DuckDB vectorized scan
Seal (async)	1–10 s	Arrow IPC $\rightarrow$ Parquet + S3 PUT	pg_ducklake: $5.8\times$ faster than vanilla DuckDB+DuckLake
Cold restart	< 30 s at 1 PB	Bounded log replay	To validate in Phase-0 benchmark
Failover	< 1 s	Follower already has OPEN log	Raft leader election

Table 11.1: Performance budgets. Targets, not guarantees.

11.1 Scale targets (honest)

Scale	Sealed data on S3	OPEN objects (bounded)	Hot catalog cache	Status
1 PB	$\sim$ 1 PB	$\sim$ 10 MB – 1 GB	$\sim$ 1–10 GB	v1 target
10 PB	$\sim$ 10 PB	$\sim$ 100 MB – 10 GB	$\sim$ 10–50 GB	v1 target (sharded)
100 PB	shard by database	shard by database	shard by database	Research

Table 11.2: Scale targets. Architecture permits large-scale deployments; practical limits depend on implementation quality in metadata scaling, compaction, and distributed execution.

11.2 Commit rate targets (honest)

v1 targets tens of thousands of commits/sec per database, scaling linearly with database count (each database's root pointers live in one Raft group). Millions of commits/sec is research — it requires hierarchical namespaces, append-only pointer logs, or commit groups, none of which are in v1.

12. Dependency Budget (Not LOC Budget)

An earlier draft of this RFC specified a lines-of-code budget for the storage kernel (< 2000 lines), the Coordinator (< 1000 lines), the Planner (< 500 lines), and the IR core (< 3000 lines). On review, LOC budgets are the wrong discipline: 500 ugly lines may be worse than 900 beautiful lines, and a clear refactor that grows LOC by 20% may be strictly better than what it replaced. The right discipline is a dependency budget — what each component may depend on, architecturally. Dependencies are enforceable by code review and tooling; LOC is not.

12.1 The dependency budget (Specified)

Component	May depend on	May NOT depend on
Storage kernel	The 4 syscalls. The lifecycle state machine. The DAG pattern. The root pointer namespace.	SQL. The Planner. The Coordinator. Any capability plugin. Any specific replication algorithm (Raft is a plugin). Any specific format (Parquet is a plugin).
Coordinator	Graph execution. Transaction boundaries. Capability discovery.	SQL. The Planner. Any specific capability. Cost estimation. Scheduling (backends schedule their own work).
Planner	Pass execution. Dependency ordering. Invariant verification.	Cost estimation (passes do). Backend selection (passes do). Routing decisions (passes do). Any specific capability.
Pond IR core	The 15 operators. The type system. The 5 optimization invariants. The lowering contract.	Any backend's native IR. Any specific capability. SQL parsing (wire capability).

Table 12.1: Dependency budget for each core component.

12.2 Enforcement

The dependency budget is enforced by code review and by a lint tool that checks imports. A PR that adds a forbidden dependency to a core component is blocked. If a component genuinely needs a new dependency, the PR must either (a) extract the dependent code to a plugin, or (b) justify why the dependency budget should be amended (rare; requires architecture review). This is the Linux-kernel-ABI discipline: the core is protected from growth by structural enforcement, not by goodwill.

12.3 LOC as a guideline, not a rule

LOC is tracked as a guideline. The storage kernel is expected to be roughly 2000 lines, the Coordinator roughly 1000, the Planner roughly 500, the IR core roughly 3000. If a component significantly exceeds its guideline, the team investigates whether complexity has migrated in — but the component is not blocked solely on LOC. The dependency budget is the hard rule; LOC is the soft signal.

13. Open Questions & Research

The following items are Research: acknowledged unsolved, named honestly rather than pretended solved. They are not in v1. They are the multi-year frontier where Pond will either mature into a serious production system or remain a sound architecture on paper. Each is paired with the reason it is hard and the direction a solution might take.

Research item	Why it is hard	Direction
Hierarchical namespaces for millions of commits/sec	The root pointer namespace becomes a hotspot at very high commit rates. Single Raft group tops out at ~50K commits/sec.	Hierarchical namespaces (root pointers are themselves content-addressed trees, cached aggressively). Append-only pointer logs. Commit groups (multiple txns share one commit record).
Storage-aware replication protocol	Raft replicates log entries; immutable object fragments could be replicated more efficiently (content-addressed, K-of-N, no leader). This is the genuine novel research contribution.	Content-addressed fragments (hash = address, like Git blobs). K-of-N quorum writes (any node accepts). Gossip-based membership. Erasure coding for sealed objects. Geographic locality. Closer to IPFS + Cassandra + BitTorrent than to Raft.
Distributed execution (Exchange across nodes)	The Exchange operator exists in the IR but v1 runs graphs single-node. Multi-node execution requires shuffle, skew handling, locality, adaptive repartition, spilling, backpressure.	v2+. IR supports it; runtime does not yet. Spark, Flink, Materialize all spent years here. Pond will too.
Cross-backend cost-based planning	Choosing between DuckDB, GPU, Velox, remote node, etc. for a subgraph requires a sophisticated optimizer. Nobody has solved this well; Spark Catalyst, Calcite, DuckDB optimizer all do cost-based within one engine, not across engines.	Learned cost models (plugins contribute cost functions). Adaptive estimation. Exhaustive search with pruning. Multi-year effort.
Streaming semantics (exactly-once, watermarks, late data)	Materialize spent years on differential dataflow. Flink spent years. Timely Dataflow spent years. Streaming is genuinely hard research.	v1 supports batch + simple streaming (poll-based tail reads). Full differential-dataflow-class streaming is multi-year research. Honestly acknowledged.

Research item	Why it is hard	Direction
Learned indexes	Indexes that learn access patterns and restructure themselves. Research, not production-proven at lakehouse scale.	Future index/learned.so plugin. The Index trait is Specified; implementations are plugins.
Operator fusion and adaptive optimization	Runtime feedback that fuses operators and re-plans mid-query. Umbra territory. Requires morsel-driven execution.	Future compute plugins. The architecture permits it; the implementation is years away.

Table 13.1: Research items. Acknowledged unsolved; multi-year frontier.

13.1 What is explicitly NOT claimed

Pond does not claim to beat DuckDB at OLAP — DuckDB IS the reference backend. Pond does not claim TigerBeetle-class mechanical sympathy — measurable budgets are the target, and the default DuckDB backend may not meet TigerBeetle's bar (future backends might). Pond does not claim to subsume Spark, Flink, Kafka, or Databricks — it is defined by what it IS (a capability-oriented data runtime), not by what it replaces. Pond does not claim the storage-aware replication protocol is solved — it is named as research. Pond does not claim PB-scale is proven — the architecture permits it; the implementation must demonstrate it.

14. References & Cross-References

14.1 The other four RFCs

This is RFC 1 of 5. The complete architecture is specified across five documents:

RFC	Title	Status
RFC 1	Storage & Versioned State (this document)	v1.0, architecture-frozen
RFC 2	Planner & IR Specification	Planned
RFC 3	Transaction & Consistency Model	Planned
RFC 4	Capability & Placement Model	Planned
RFC 5	Operational Architecture	Planned

14.2 Projects that informed this design

Pond's architecture is a synthesis of ideas from many systems. The synthesis is original; the individual ideas are not. Credit where credit is due:

Project	What Pond borrowed	What Pond rejected
Git	Content-addressed DAG. blob/tree/commit/tag. Immutable history.	NoSQL. No transactions. No concurrency.
LLVM	IR + lowering contract. PassManager model. Backend neutrality. IR stability across versions.	Compiler-specific abstractions.
Linux	Syscall ABI stability. Kernel/userspace separation. Driver model. Dependency budget discipline.	OS-level process scheduling.
SQLite	Embedded, single-file philosophy. Public-domain stability promise.	Single-writer. No scale-out.
TigerBeetle	Deterministic state machine replication. Static memory. Cache-line-aligned records. Viewstamped Replication.	Fixed schema. No SQL. Single-shard only.
FoundationDB	Log-as-recovery-buffer pattern. Sequencer. Layered SQL on ordered KV. Non-blocking OCC.	KV-only data model.
DuckDB	Vectorized execution. SQL optimizer. Extension API. ART index. In-process embeddability.	Single-process-write assumption. No streaming SQL.
DuckLake v1.0	SQL catalog + Parquet on object storage. Data inlining. Multi-writer via catalog DB.	n/a (foundational).
pg_ducklake	Postgres access method pattern. Background maintenance worker.	Tying to Postgres (Pond uses DuckDB-native catalog by default).
pg_duckpipe	Internal delta-processor pattern. CREATE PIPESOURCE DX.	n/a (foundational).
ClickHouse	Sparse primary indexes. MergeTree compaction. SIMD scans.	MergeTree-as-format (we use Parquet). MergeTree-as-engine (too monolithic).
PocketBase	Single-binary philosophy. Embedded or server. Zero config.	Single-node only.
Apache Iceberg	Open table format. REST catalog spec. Snapshot semantics.	Manifest explosion at PB scale. Catalog-as-separate-service.
Databricks LTAP	The "one copy" problem statement. The agent-native motivation.	Two durable copies (Postgres + Iceberg). Minutes-of-sync-lag reality.
Apache Fluss	OPEN/SEALED streaming intuition. Sub-second freshness target.	ZooKeeper dependency. Tablet/ISR machinery. JVM.

Table 14.1: Influences. Synthesis is original; individual ideas are not.

14.3 The unique contribution

Pond. Pond is a capability-oriented data runtime: a minimal core coordinating many execution strategies over a single versioned, immutable, one-copy object substrate, with everything staying local unless coordination is provably necessary. **One copy. Infinite execution. Zero coordination unless necessary.**

No existing system has this exact combination: one canonical copy, capability composition, a tiny coordinating core, local-first operation, and an architectural commitment to removing concepts rather than adding them. The individual pieces exist in many systems; the synthesis does not. If Pond succeeds, it will be because the synthesis holds — not because any single piece is novel.

14.4 Document status

This document is the formal specification of Pond's storage and Versioned State layer. It is intended to be precise enough that two independent implementations of the Specified behaviors would produce identical observable results. Implementation-defined behaviors are flagged; research items are acknowledged. Changes to Specified behaviors require a new version of this RFC and a migration plan. Changes to Implementation-defined behaviors do not. Changes to Research items are the expected output of future work.

End of RFC 1. RFC 2 (Planner & IR Specification) follows.